Software Requirements Specification

For

Fresh Cart(Full-Stack)

Prepared by:

Nada Ahmed Ezzat 20230625

Hania Hisham Ahmed 20230638

Hannah Ramadan Mohamed 20230646

Yara Hossam Elden Wahed 20230656

Amr Salah Ebrahim 20230384

Muhammad Walid Muhammad Zaky 20230515

Organization : Helwan University

Functional Requirements

User Requirements	System Requirements
Users can register for a new account.	The system shall provide a user registration module with secure data storage.
Users can log in with different roles (admin, customer, guest).	The system shall implement a role-based access control system with secure login functionalities.
Users can log out of their account to terminate the session.	The system shall provide secure session management and termination capabilities.
Users can forget/reset their password.	The system shall provide a forget-password flow with verification code and password reset.
Users can change their password.	The system shall allow authenticated users to update their password securely.
Users can view and manage their profile.	The system shall provide profile management with update user role capabilities for admins.
Admin can view all users.	The system shall provide an admin panel listing all registered users.
Admin can add or delete a user.	The system shall allow admins to add new users and remove existing ones.
Admin can update a user's role.	The system shall allow admins to assign or change roles (admin/customer) for any user.
Users can view all products.	The system shall display a complete product catalog with search and filter capabilities.
Users can view product details.	The system shall display detailed product information including price, images, and description.
Admin can add a product.	The system shall provide a product creation interface for admins.
Admin can update a product.	The system shall allow admins to edit product information and availability.
Admin can delete a product.	The system shall allow admins to permanently remove products from the catalog.

User Requirements	System Requirements
Users can view all categories.	The system shall display all product categories available on the platform.
Users can get a specific category.	The system shall allow filtering and retrieval of products by a specific category.
Users can view all subcategories.	The system shall display all subcategories under each main category.
Users can get subcategories on a category.	The system shall allow retrieval of subcategories linked to a selected category.
Users can view all brands.	The system shall display all brands available on the platform.
Users can get a specific brand.	The system shall allow users to filter products by a specific brand.
Customers can add a product to cart.	The system shall provide an add-to-cart mechanism linked to the user's session.
Customers can view their cart.	The system shall display all cart items with quantities and total price.
Customers can update cart product quantity.	The system shall allow customers to change the quantity of items in the cart.
Customers can delete a product from cart.	The system shall allow removal of individual items from the cart.
Customers can clear the entire cart.	The system shall provide an option to empty the entire cart at once.
Customers can make an order.	The system shall process orders, validate stock, and confirm purchases with a summary.
Customers can view their own orders.	The system shall display an order history page for the authenticated customer.
Admin can list all orders.	The system shall provide admins with a complete list of all platform orders.
Customers can add products to wishlist.	The system shall maintain a wishlist collection per user account.
System can generate and refresh tokens.	The system shall issue JWT tokens on login and support token refresh for session continuity.

Nonfunctional Requirements

1 Performance Requirements

The system must handle up to 1,000 concurrent users with minimal latency.
API response time for product listings must not exceed 2 seconds under peak load.
Cart and order operations must complete within 1 second for a smooth user experience.

2 Safety Requirements

The system must ensure data integrity during all read and write operations.
All payment transactions must be atomic — partial failures must be fully rolled back.
The system must prevent duplicate orders resulting from network errors or double submissions.

3 Security Requirements

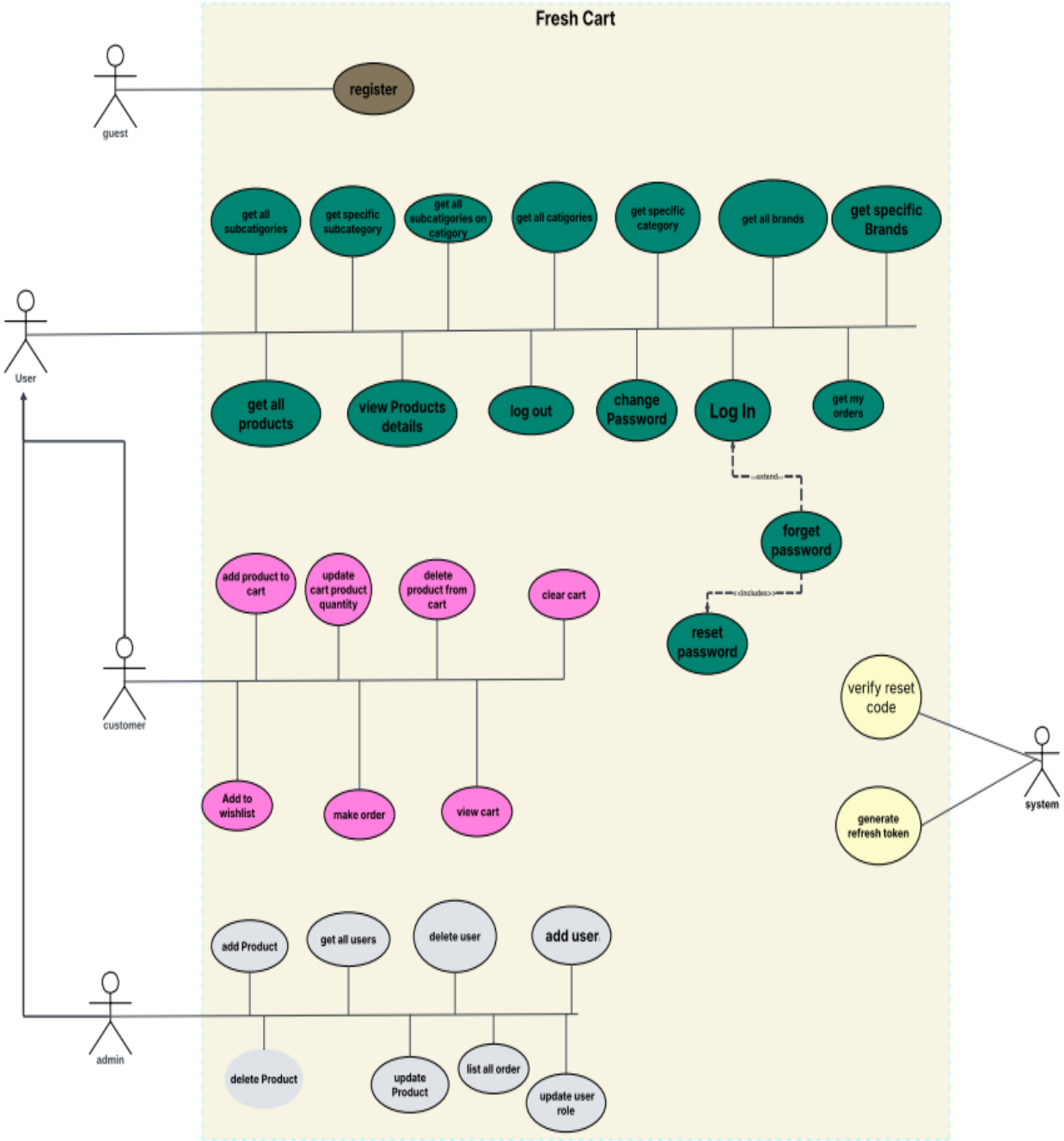
User authentication is required for all cart, order, and wishlist operations.
All user passwords must be hashed using a secure algorithm (e.g., bcrypt) before storage.
JWT tokens must be used for session management with a defined expiry and refresh mechanism.
Role-based access control (RBAC) must restrict admin routes from regular customers.
All data transmitted between client and server must be encrypted using HTTPS/TLS.

4 Software Quality Attributes

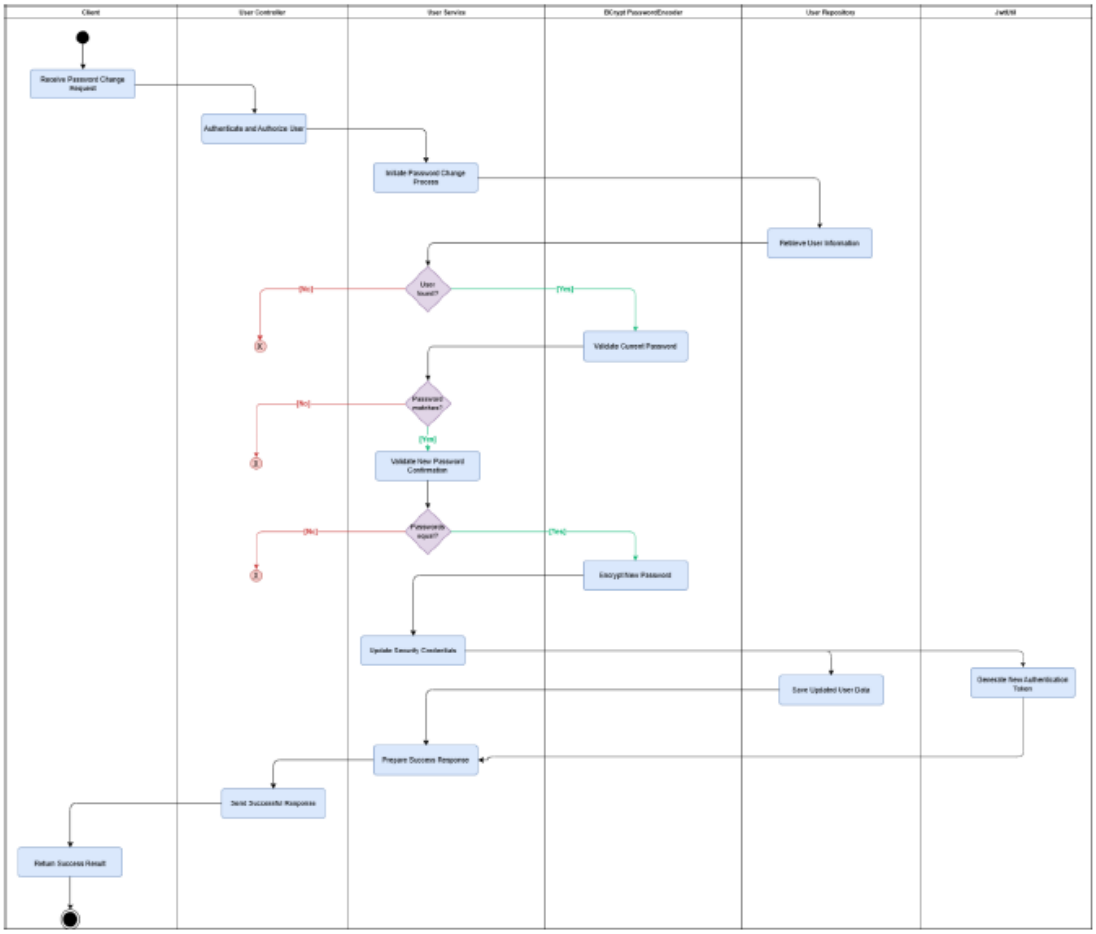
- The application must achieve 99.5% uptime (availability).
- The system must be user-friendly and responsive across all major devices and platforms.
- The codebase must be maintainable and follow RESTful API design conventions.
- The system must be scalable to support future expansion of product categories and users.

5 Business Rules

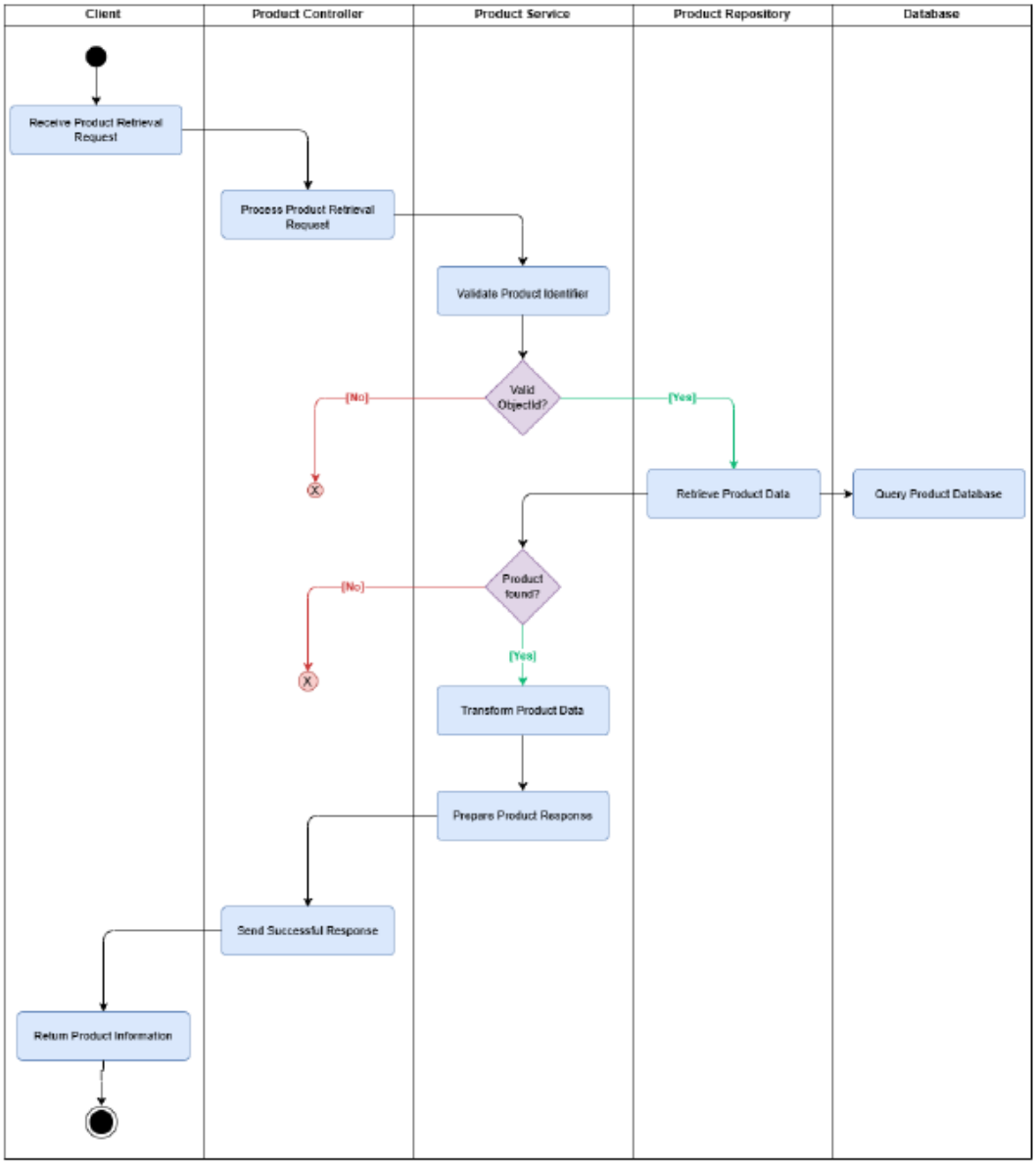
Only authenticated customers may add products to the cart, place orders, or manage wishlists.
Only admin users may create, update, or delete products, categories, brands, and users.
Guests may only browse the product catalog and view product details without authentication.
Orders may only be placed when all cart items are in stock.


Fresh Cart
guest
register
get all subcatigories
get specific subcategory
get all subcatigories on catigory
get all catigories
get specific category
get all brands
get specific Brands
User
get all products
view Products details
log out
change Password
Log In
get my orders
forget password
reset password
customer
add product to cart
update cart product quantity
delete product from cart
clear cart
Add to wishlist
make order
view cart
verify reset code
generate refresh token
system
admin
add Product
get all users
delete user
add user
delete Product
update Product
list all order
update user role

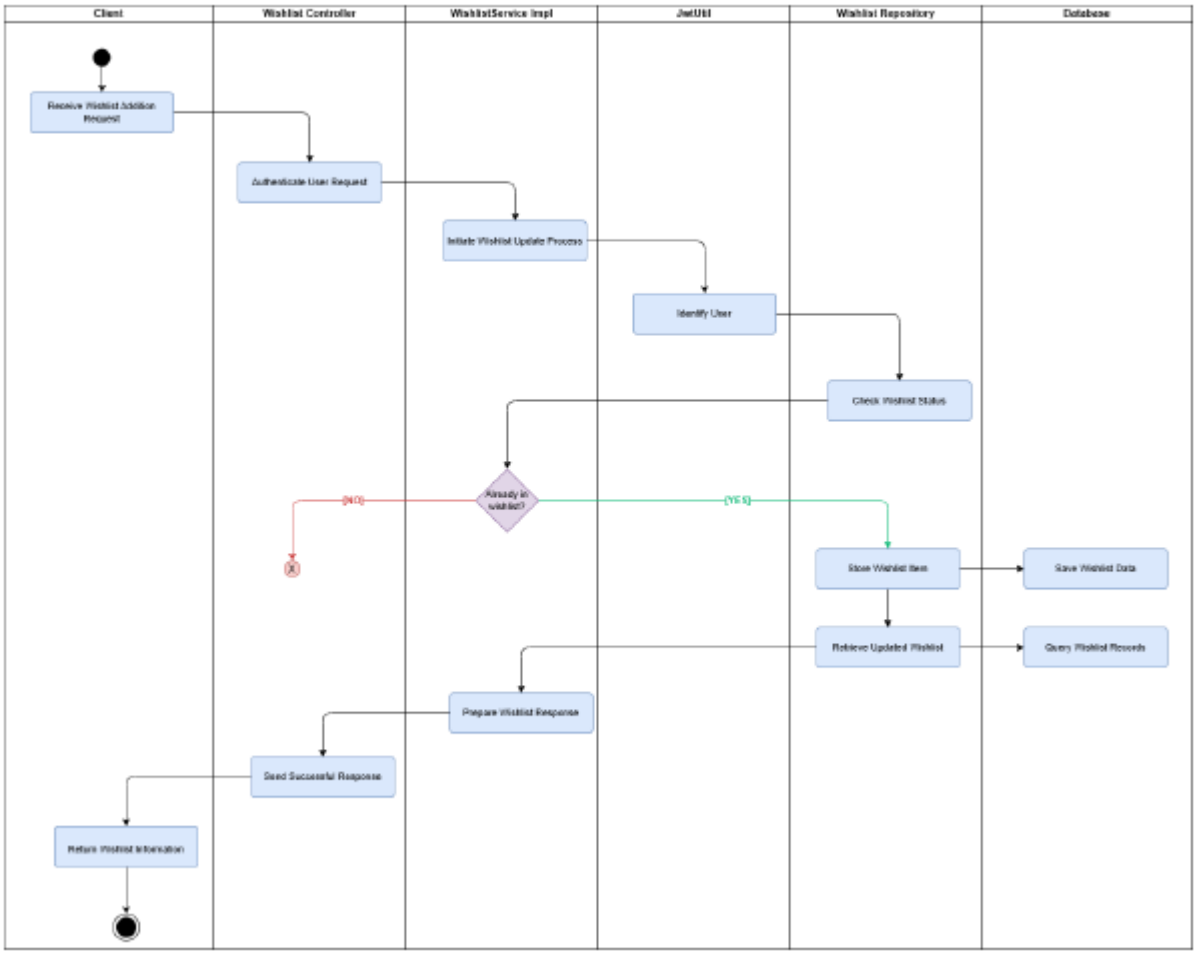
User Service



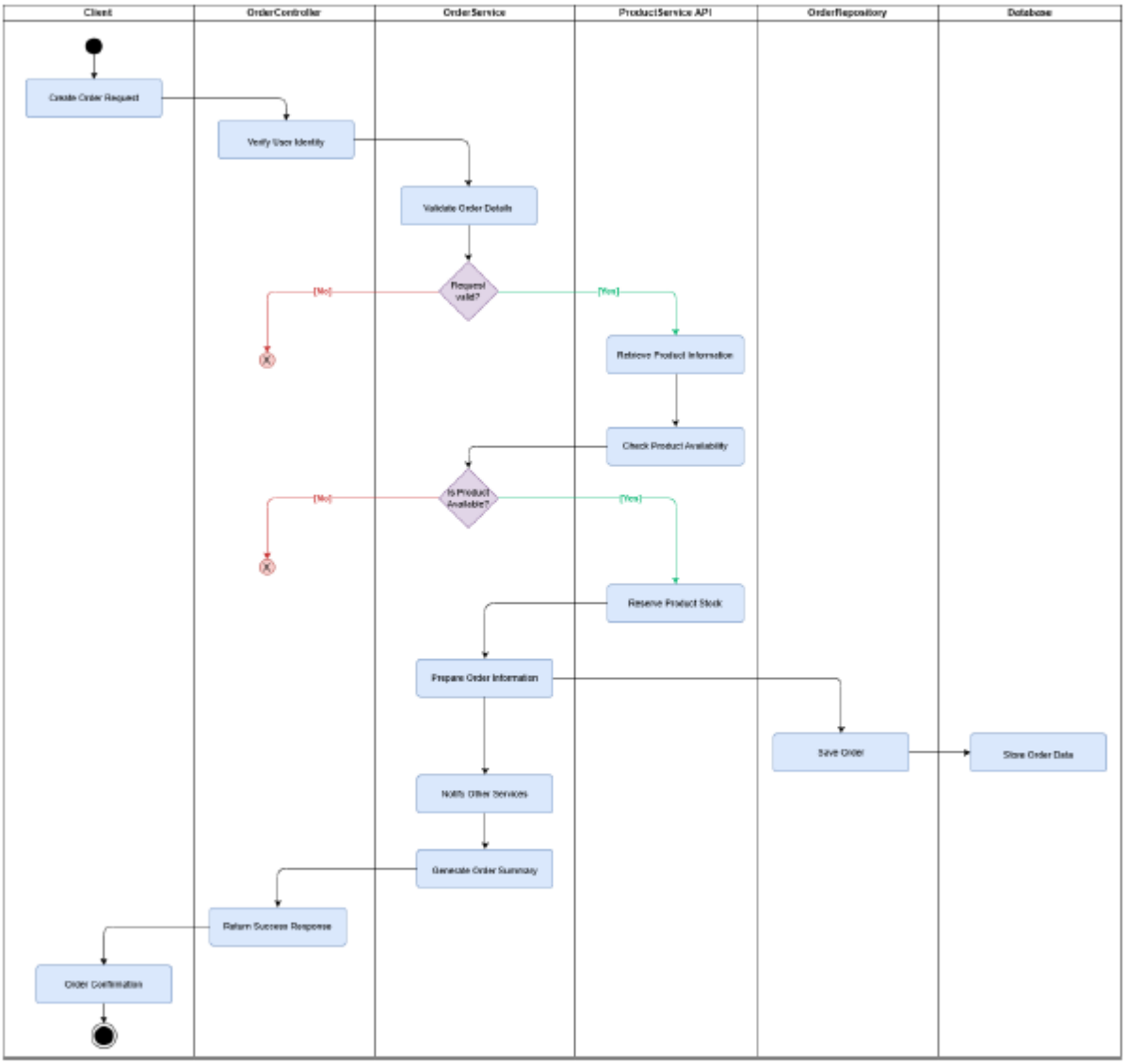
Product Service



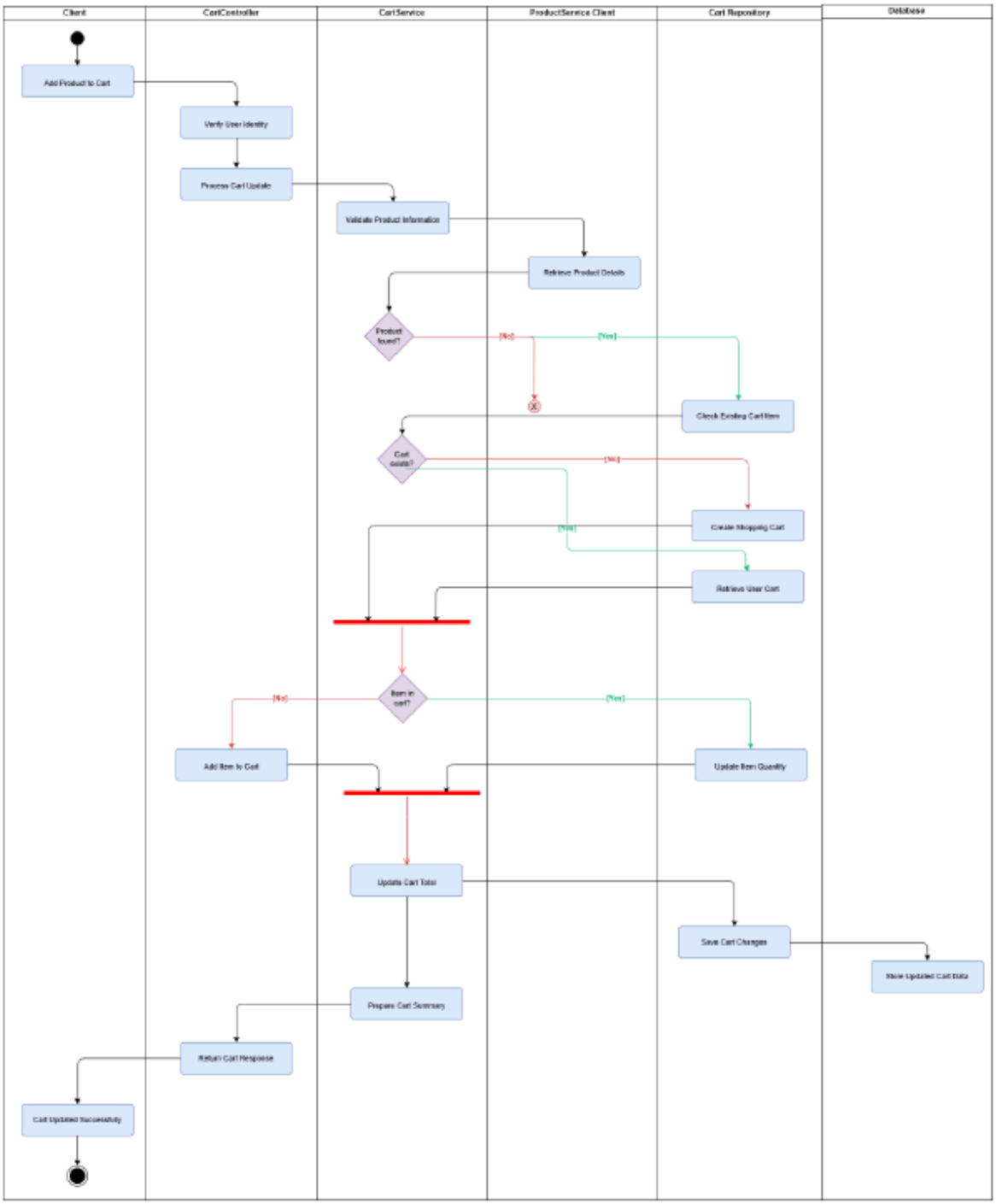
Wishlist Service



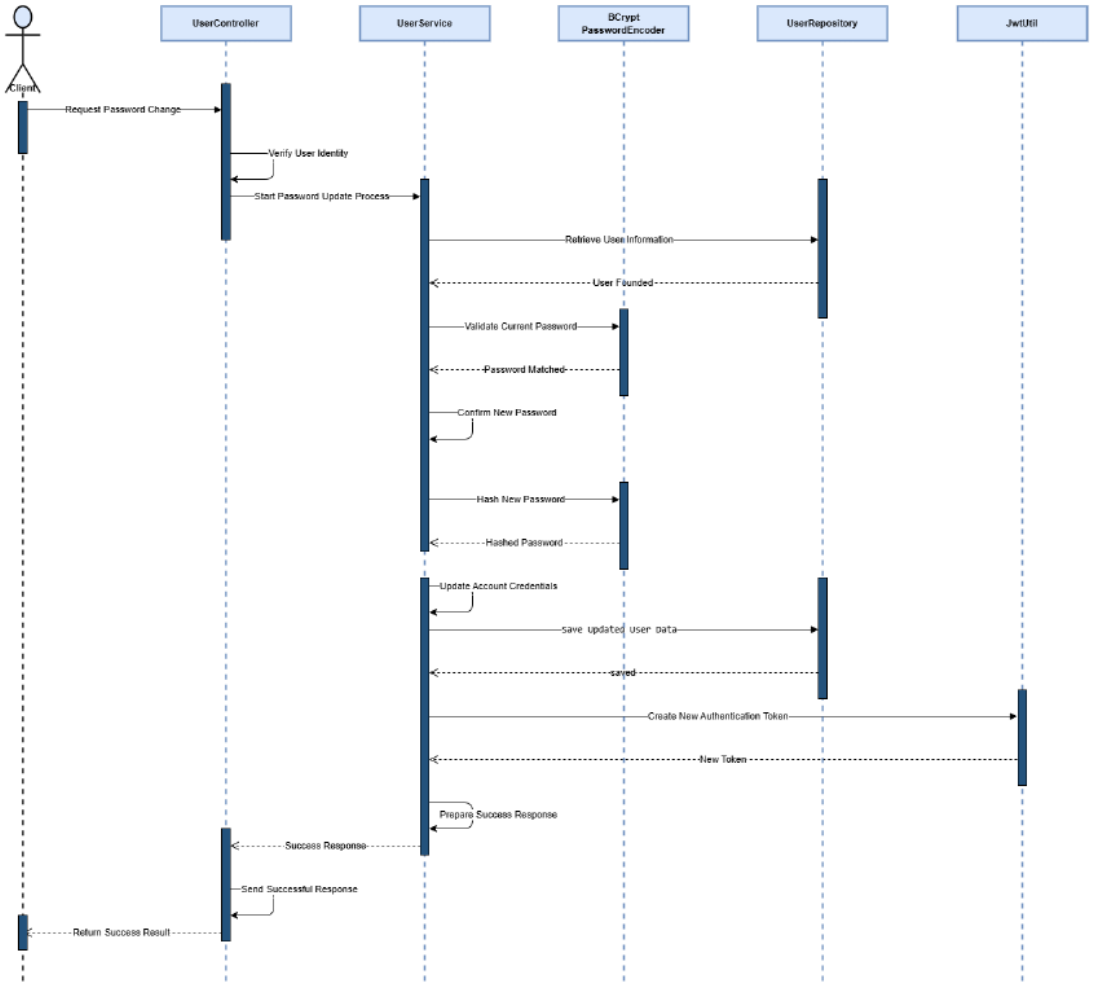
Order Service



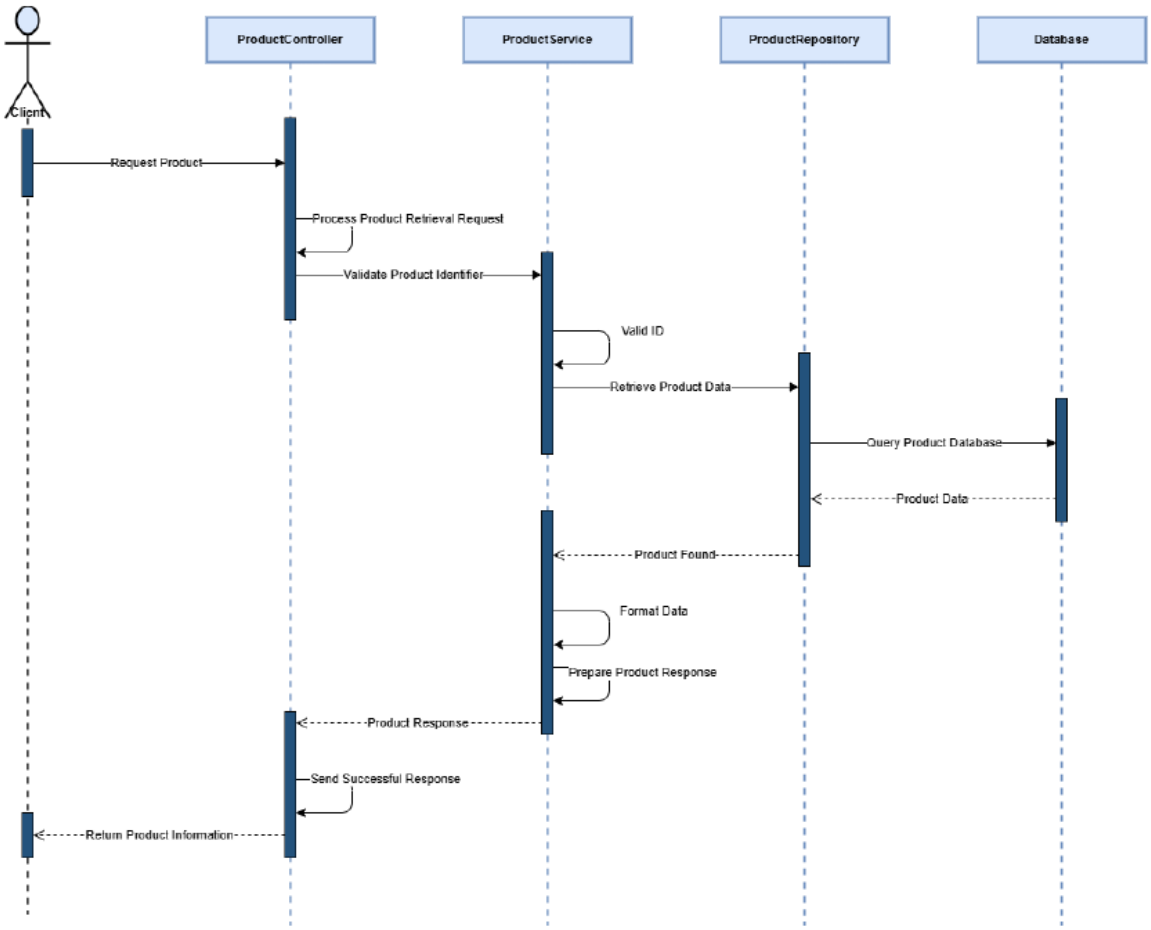
Cart Service



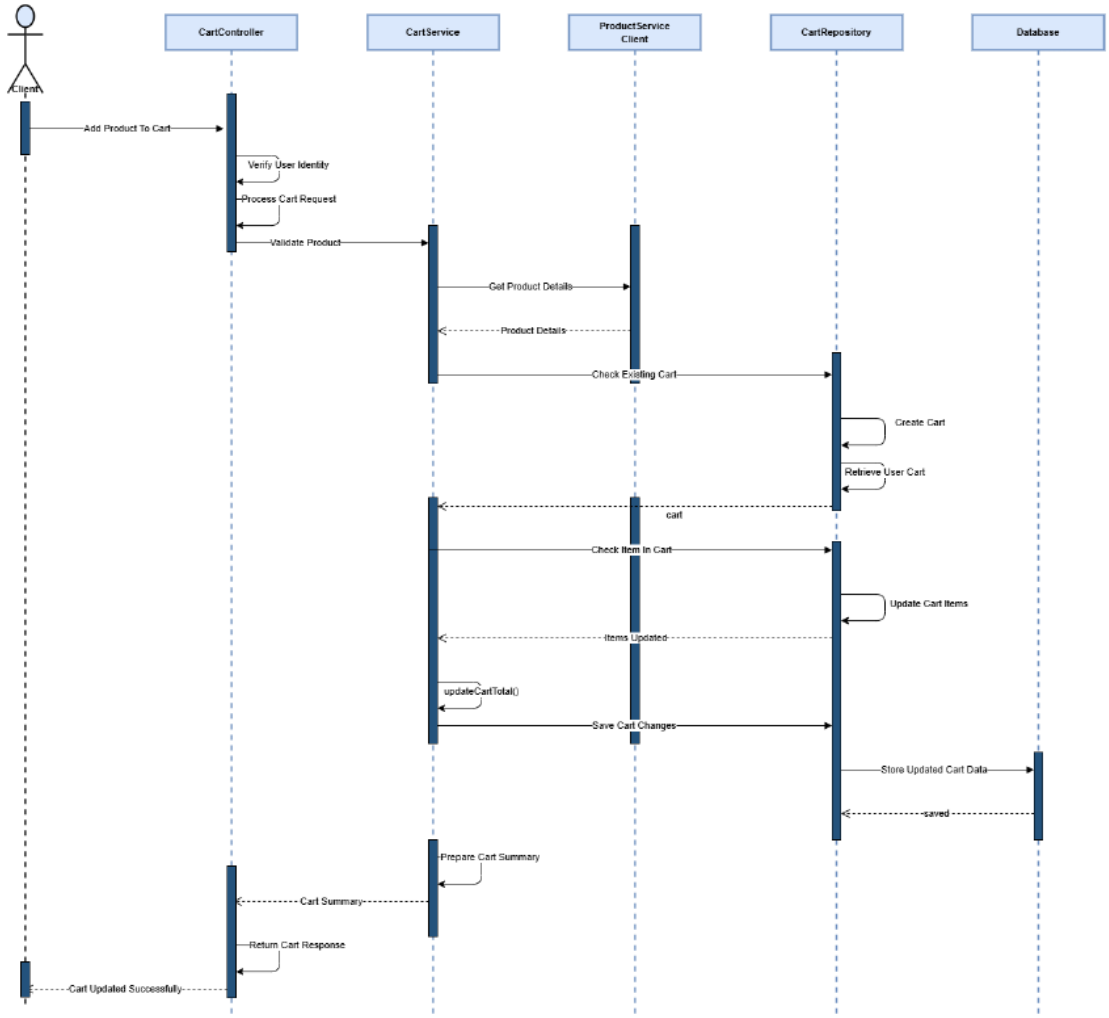
User Service



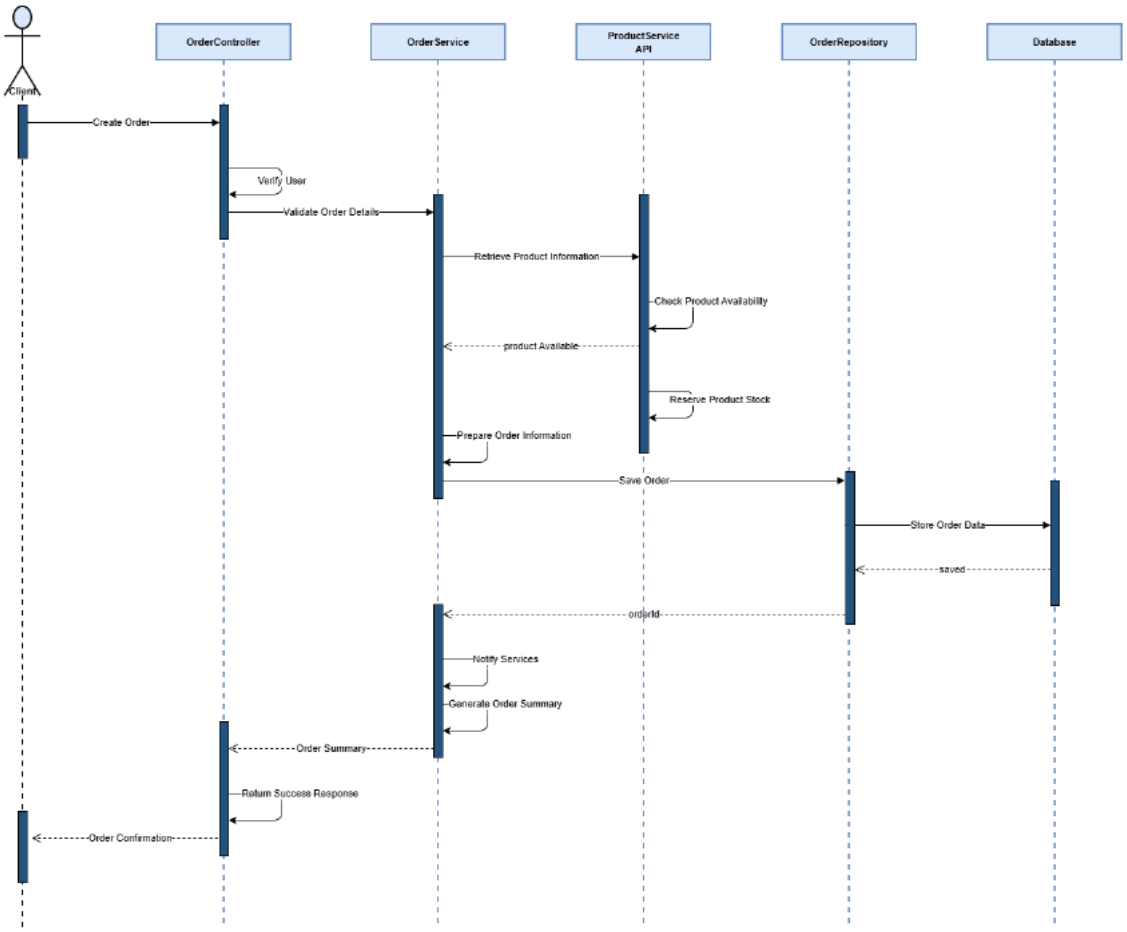
Product Service



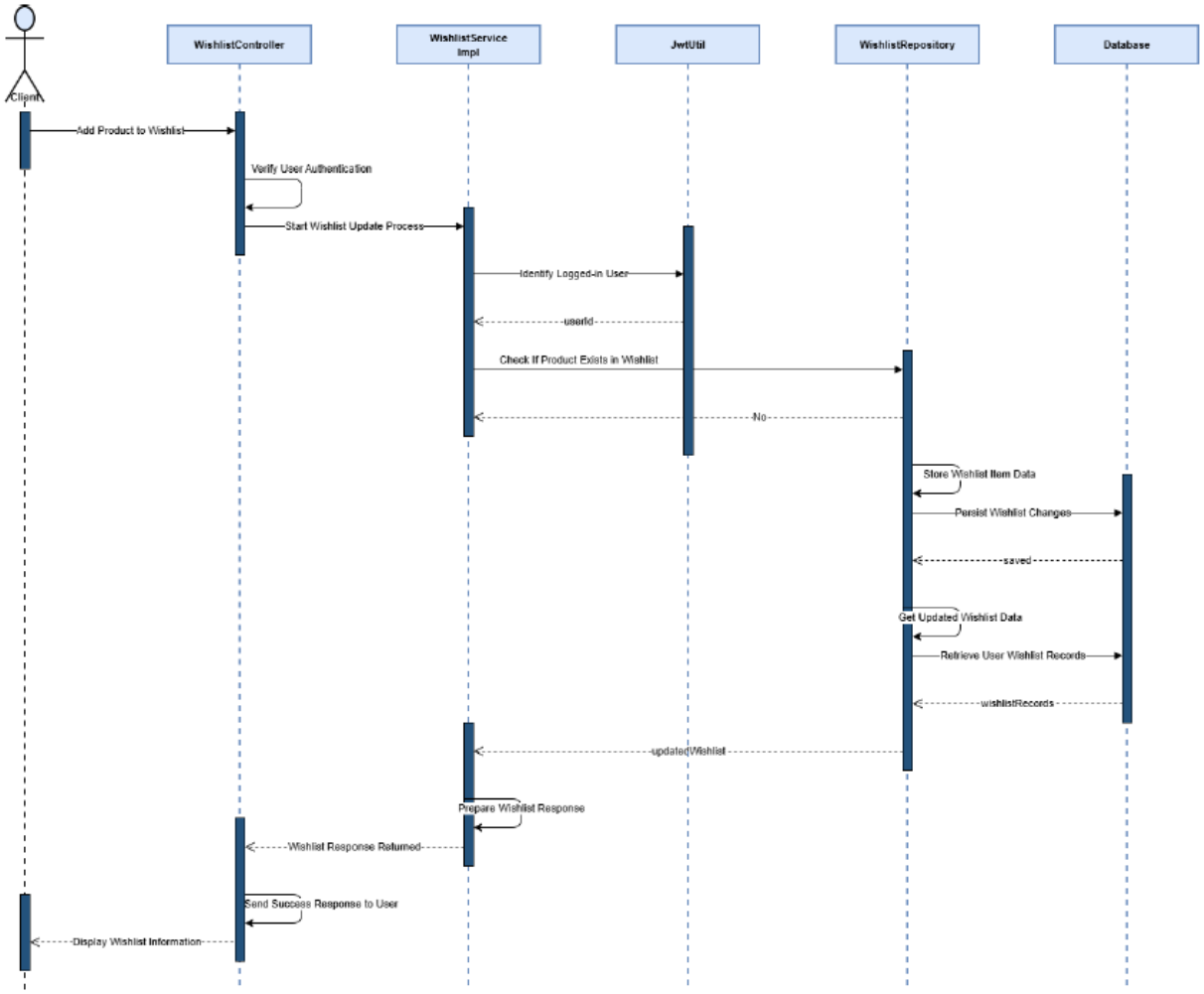
Cart Service



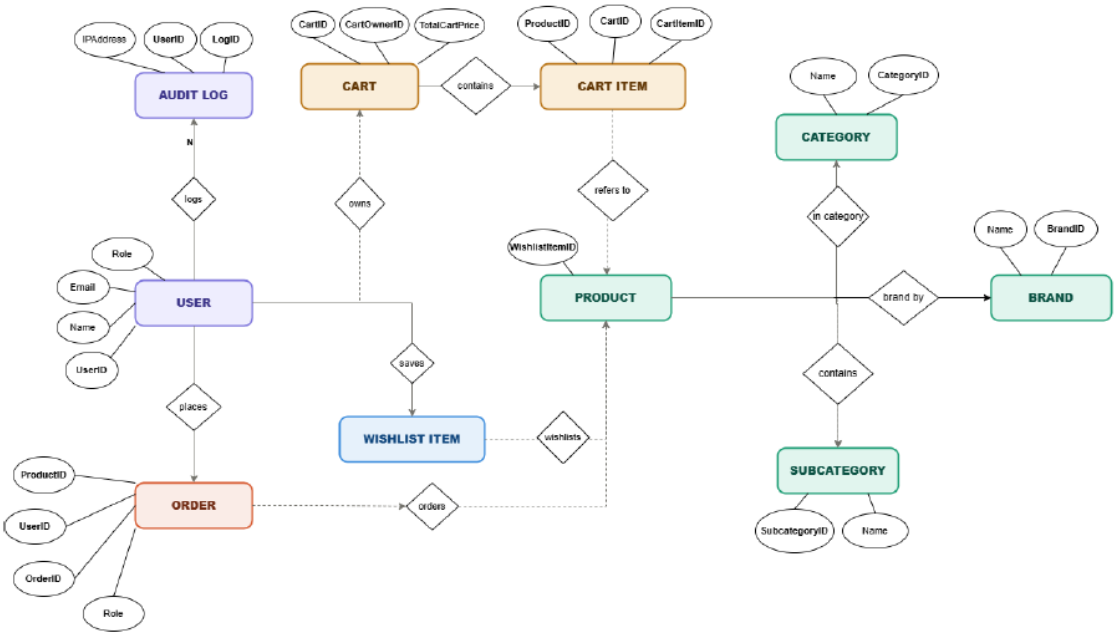
Order Service



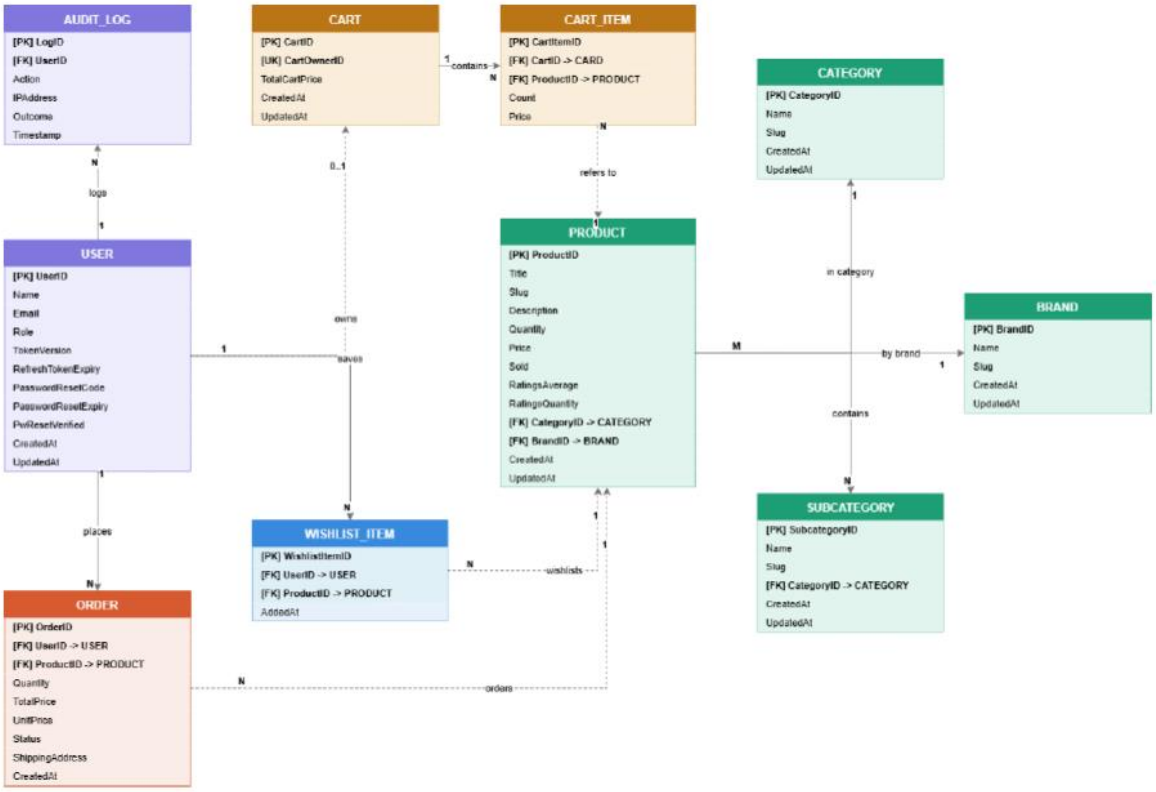
Wishlist Service



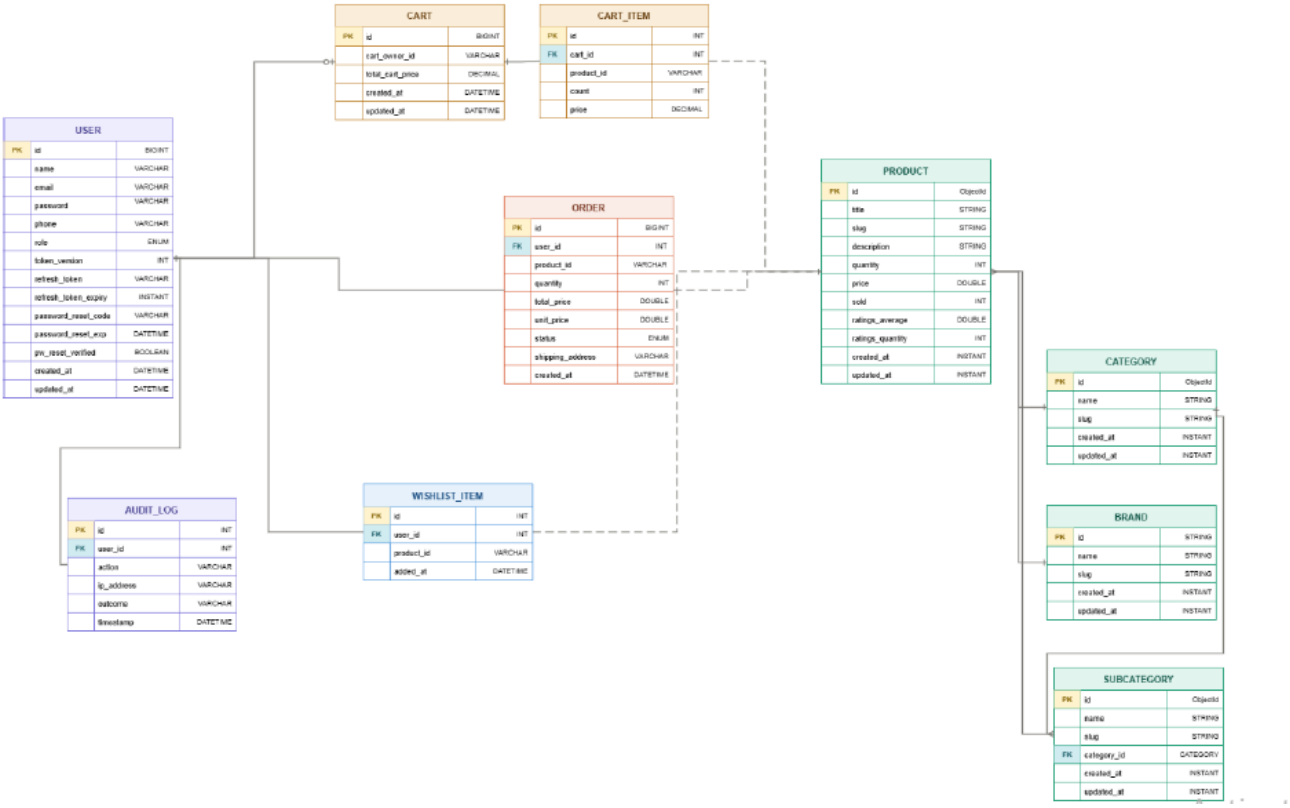
Conceptual ERD



Logical ERD


AUDIT_LOG
[PK] LogID
[FK] UserID
Action
IPAddress
Outcome
Timestamp
CART
[PK] CartID
[UK] CartOwnerID
TotalCartPrice
CreatedAt
UpdatedAt
CART_ITEM
[PK] CartItemID
[FK] CartID -> CARD
[FK] ProductID -> PRODUCT
Count
Price
CATEGORY
[PK] CategoryID
Name
Slug
CreatedAt
UpdatedAt
USER
[PK] UserID
Name
Email
Role
TokenVersion
RefreshTokenExpiry
PasswordResetCode
PasswordResetExpiry
PwResetVerified
CreatedAt
UpdatedAt
PRODUCT
[PK] ProductID
Title
Slug
Description
Quantity
Price
Sold
RatingsAverage
RatingsQuantity
[FK] CategoryID -> CATEGORY
[FK] BrandID -> BRAND
CreatedAt
UpdatedAt
BRAND
[PK] BrandID
Name
Slug
CreatedAt
UpdatedAt
WISHLIST_ITEM
[PK] WishlistItemID
[FK] UserID -> USER
[FK] ProductID -> PRODUCT
AddedAt
SUBCATEGORY
[PK] SubcategoryID
Name
Slug
[FK] CategoryID -> CATEGORY
CreatedAt
UpdatedAt
ORDER
[PK] OrderID
[FK] UserID -> USER
[FK] ProductID -> PRODUCT
Quantity
TotalPrice
UnitPrice
Status
ShippingAddress
CreatedAt
contains
refers to
logs
owns
saves
places
in category
by brand
contains
wishlists
orders

Physical ERD



Design Patterns

1. BUILDER DESIGN PATTERN

Context:

The Fresh Cart system constructs complex objects throughout its microservices: Cart and Order domain entities with many optional and required fields, response DTOs (OrderData, UserSummary, OrderItemResponse) assembled in mapper layers, and snapshot objects. These objects have multiple fields that vary by scenario, making telescoping constructors error-prone and unreadable.

Problem:

How can we construct complex multi-field objects (Cart, Order, OrderData, UserSummary, OrderItemResponse) in a readable, step-by-step manner without requiring a constructor for every field combination, and without exposing intermediate inconsistent states?

Forces:

- Domain entities (Cart, Order) and their DTOs have many fields — some optional, some required — making constructor overloading unmaintainable.
- Mapper classes (CartMapper) must assemble response objects in a controlled, readable pipeline that reads like a specification, not a list of positional arguments.
- The construction process must be separated from the representation so the same builder pattern can produce different representations (e.g., full OrderData vs. lightweight UserSummary).

Solution:

1. Apply Lombok @Builder to all entities and DTOs: cartservice/.../entity/Cart.java, orderservice/.../model/Order.java, and response objects (OrderData, UserSummary, OrderItemResponse).
2. In CartMapper (cartservice/.../service/CartMapper.java), use the builder chain to assemble response objects field by field, making the construction intent explicit.
3. In OrderMapper, buildUserSummary(Order order) builds a minimal UserSummary via UserSummary.builder()._id(...).name(...).email(...).phone(...).build(), extracting only the fields needed for the response.
4. toItemResponse(OrderItem item) builds an OrderItemResponse via OrderItemResponse.builder()._id(...).count(...).price(...).product(...).build(), with inline conditional logic (productObj != null ? productObj : item.getProductId()) applied at field-setting time.
5. The top-level OrderData is built in one fluent chain (OrderData.builder().id(...).numericId(...).user(...).shippingAddress(...).cartItems(...) ... .build()), making field assignment auditable at a glance.

Antipatterns:

- Using telescoping constructors (e.g., new OrderData(id, numericId, user, address, items, ...)) — fragile, unreadable, and error-prone when field order changes.
- Setting fields one by one via public setters after construction, creating objects that exist in partially initialized, inconsistent states.
- Mixing builder logic with business logic inside the builder itself (builders should only construct, never validate or compute).

References:

- GoF: Design Patterns: Elements of Reusable Object-Oriented Software — Builder Pattern
- Effective Java (3rd ed.) — Item 2: Consider a builder when faced with many constructor parameters

- Lombok @Builder documentation: https://projectlombok.org/features/Builder

---

2. DEPENDENCY INJECTION / IoC DESIGN PATTERN

Context:

Fresh Cart is a Spring Boot microservices system. Each service (order-service, cart-service, user-service, product-service) contains controllers, service classes, repositories, mappers, and configuration beans. These components depend on each other — for example, an OrderService depends on an OrderRepository, a CartClient, a ProductClient, and an OrderMapper. Creating and wiring these dependencies manually would couple the code tightly and make testing nearly impossible.

Problem:

How can we decouple the creation and lifecycle of service dependencies (repositories, clients, mappers, config values) from the business logic classes that use them, allowing components to be swapped, mocked in tests, and configured externally?

Forces:

- Service classes (e.g., OrderService, CartService) must not instantiate their own dependencies; doing so hardcodes implementation choices and prevents unit testing.
- Configuration values (e.g., Stripe keys, JWT secrets, service URLs) must be injected from environment/config files rather than hardcoded.
- The system must support constructor injection as the preferred style so dependencies are explicit, final, and the class is always in a valid state after construction.
- Cross-cutting components (security filters, serializers, Feign clients) must be registered as Spring beans and made available throughout the application context.

Solution:

6. Apply Spring stereotype annotations throughout all services and controllers: @Service on business logic classes, @Component on utility/mapper classes, @RestController on API layers, and @Configuration on config classes.
7. Use constructor injection (the recommended Spring practice) for all mandatory dependencies — Lombok @RequiredArgsConstructor or explicit constructors — so fields can be declared final and are guaranteed non-null at runtime.
8. Register Feign clients, RestTemplate, WebClient, and other infrastructure beans in @Configuration classes so they are managed by the Spring IoC container and can be injected wherever needed.
9. Inject environment-specific values (JWT secret, expiry times, external service URLs) via @Value('${property.key}') or @ConfigurationProperties, keeping configuration out of business logic.
10. Spring's ApplicationContext acts as the IoC container — it instantiates, wires, and manages the full lifecycle of all beans across all microservices, enabling the Inversion of Control principle: classes declare what they need, and the container provides it.

Antipatterns:

- Using field injection (@Autowired directly on fields) — it hides dependencies, makes classes harder to test, and prevents declaring fields as final.
- Instantiating dependencies with new inside service classes (e.g., new OrderRepository()) — this defeats IoC and tightly couples the class to a specific implementation.
- Placing business logic inside @Configuration classes or @Bean methods — configuration should only wire components, never contain domain rules.

- Circular dependencies between beans — e.g., ServiceA depends on ServiceB which depends on ServiceA — should be resolved by extracting shared logic into a third component.

References:

- Spring Framework Documentation: Core — IoC Container (https://docs.spring.io/spring-framework/docs/current/reference/html/core.html)
- GoF: Design Patterns: Elements of Reusable Object-Oriented Software — (related: Factory, Strategy)
- Effective Java (3rd ed.) — Item 5: Prefer dependency injection to hardwiring resources

Use Cases Description

Use Case 1: Register

Identifier and Name	UC1 - Register
Initiator	Guest / New User
Goal	The user creates a new account to access the Fresh Cart platform.
Precondition	The user has a valid email address and internet connection.
Postcondition	The user account is created and stored securely in the system.
Assumptions	The user provides valid and unique credentials.

Main Success Scenario:

1. The user navigates to the registration page.
2. The user enters their name, email, and password.
3. The system validates the entered information.
4. The system stores the user data securely.
5. The system confirms successful registration.

Use Case 2: Log In

Identifier and Name	UC2 - Log In
Initiator	Guest (Admin / Customer)
Goal	The user authenticates and gains access to their account with the appropriate role.
Precondition	The user has a registered account and internet connection.
Postcondition	The user is authenticated and a session token is generated.
Assumptions	The user remembers their credentials.

Main Success Scenario:

6. The user navigates to the login page.
7. The user enters their email and password.
8. The system verifies the credentials against the database.
9. The system generates a JWT access token and refresh token.
10. The user is redirected to their role-appropriate dashboard.

Use Case 3: Log Out

Identifier and Name	UC3 - Log Out

Initiator	Authenticated User (Admin / Customer)
Goal	The user terminates their active session.
Precondition	The user is currently logged in with an active session.
Postcondition	The session is terminated and the token is invalidated.
Assumptions	The user has a stable internet connection.

Main Success Scenario:

11. The user clicks the logout button.
12. The system invalidates the user's session token.
13. The system clears any stored session data on the client.
14. The user is redirected to the login or home page.

Use Case 4: Forget Password

Identifier and Name	UC4 - Forget Password
Initiator	Unathenticated User
Goal	The user recovers access to their account by resetting their password.
Precondition	The user has a registered account and access to their email.
Postcondition	The user's password is successfully reset.
Assumptions	The user has access to the registered email address.

Main Success Scenario:

15. The user navigates to the 'Forgot Password' page.
16. The user enters their registered email address.
17. The system sends a verification/reset code to the email.
18. The user enters the reset code.
19. The system verifies the code and prompts for a new password.
20. The user enters and confirms the new password.
21. The system updates the password and confirms success.

Use Case 5: Change Password

Identifier and Name	UC5 - Change Password
Initiator	Authenticated User
Goal	The user changes their current password to a new one.
Precondition	The user is authenticated and knows their current password.
Postcondition	The user's password is updated in the system.
Assumptions	The user provides a valid current password and a strong new password.

Main Success Scenario:

22. The user navigates to the account settings page.
23. The user enters their current password.
24. The user enters and confirms a new password.
25. The system validates the current password and the new password strength.
26. The system updates the password and notifies the user.

Use Case 6: View All Products

Identifier and Name	UC6 - View All Products
Initiator	Guest / Customer / Admin
Goal	The user browses the full product catalog available on the platform.
Precondition	The user has internet access and products exist in the system.
Postcondition	The product list is displayed to the user.
Assumptions	At least one product is available in the database.

Main Success Scenario:

27. The user navigates to the products page.
28. The system retrieves all available products from the database.
29. The system displays products with name, image, price, and category.
30. The user can scroll and browse through the listings.

Use Case 7: View Product Details

Identifier and Name	UC7 - View Product Details
Initiator	Guest / Customer / Admin
Goal	The user views detailed information about a specific product.
Precondition	The product exists in the system.
Postcondition	The detailed product page is rendered for the user.
Assumptions	The product ID is valid.

Main Success Scenario:

31. The user clicks on a specific product from the catalog.
32. The system retrieves the product details using the product ID.
33. The system displays full information: name, description, images, price, brand, and category.
34. The user can decide to add the product to cart or wishlist.

Use Case 8: Add Product (Admin)

Identifier and Name	UC8 - Add Product (Admin)
Initiator	Admin
Goal	The admin adds a new product to the platform catalog.
Precondition	The admin is authenticated with admin privileges.
Postcondition	The new product is stored in the database and visible in the catalog.
Assumptions	The admin provides complete and valid product information.

Main Success Scenario:

35. The admin navigates to the product management panel.
36. The admin fills in the product form (name, price, category, brand, images, description).
37. The system validates the input data.
38. The system stores the new product in the database.
39. The system confirms successful creation and displays the new product.

Use Case 9: Update Product (Admin)

Identifier and Name	UC9 - Update Product (Admin)
Initiator	Admin
Goal	The admin modifies an existing product's information.
Precondition	The admin is authenticated and the product exists in the system.
Postcondition	The product information is updated in the database.
Assumptions	The admin provides valid updated data.

Main Success Scenario:

40. The admin selects a product from the management panel.
41. The admin edits the desired fields (price, description, stock, etc.).
42. The system validates the updated data.
43. The system saves the changes to the database.
44. The system confirms the update and reflects changes in the catalog.

Use Case 10: Delete Product (Admin)

Identifier and Name	UC10 - Delete Product (Admin)
Initiator	Admin
Goal	The admin permanently removes a product from the catalog.
Precondition	The admin is authenticated and the product exists in the system.
Postcondition	The product is removed from the database and no longer visible.

Assumptions	The admin confirms the deletion action.

Main Success Scenario:

45. The admin selects a product from the management panel.
46. The admin clicks the delete button.
47. The system prompts for confirmation.
48. The admin confirms the deletion.
49. The system removes the product from the database.
50. The system confirms successful deletion.

Use Case 11: Add Product to Cart

Identifier and Name	UC11 - Add Product to Cart
Initiator	Customer
Goal	The customer adds a selected product to their shopping cart.
Precondition	The customer is authenticated and the product is in stock.
Postcondition	The product is added to the customer's cart with the selected quantity.
Assumptions	The product is available and the customer has an active session.

Main Success Scenario:

51. The customer views a product detail page.
52. The customer selects quantity and clicks 'Add to Cart'.
53. The system validates product availability.
54. The system adds the product to the customer's cart in the database.
55. The system updates the cart count and confirms the action.

Use Case 12: View Cart

Identifier and Name	UC12 - View Cart
Initiator	Customer
Goal	The customer views all products currently in their cart.
Precondition	The customer is authenticated.
Postcondition	The cart contents are displayed with quantities and totals.
Assumptions	The customer has at least one item in the cart.

Main Success Scenario:

56. The customer navigates to the cart page.
57. The system retrieves cart items linked to the customer's account.

58. The system displays all cart items with product details, quantities, and total price.

Use Case 13: Update Cart Product Quantity

Identifier and Name	UC13 - Update Cart Product Quantity
Initiator	Customer
Goal	The customer modifies the quantity of a product in their cart.
Precondition	The customer is authenticated and the product is in the cart.
Postcondition	The cart is updated with the new quantity and recalculated total.
Assumptions	The requested quantity is available in stock.

Main Success Scenario:

59. The customer opens their cart.
60. The customer changes the quantity field of a product.
61. The system validates the new quantity against stock availability.
62. The system updates the cart item quantity in the database.
63. The system recalculates and displays the updated total.

Use Case 14: Delete Product from Cart

Identifier and Name	UC14 - Delete Product from Cart
Initiator	Customer
Goal	The customer removes a specific product from their cart.
Precondition	The customer is authenticated and the product exists in the cart.
Postcondition	The product is removed from the cart and totals are recalculated.
Assumptions	The customer confirms the removal.

Main Success Scenario:

64. The customer opens their cart.
65. The customer clicks the remove button next to a product.
66. The system removes the item from the cart in the database.
67. The system updates and displays the revised cart total.

Use Case 15: Clear Cart

Identifier and Name	UC15 - Clear Cart
Initiator	Customer
Goal	The customer empties their entire shopping cart at once.

Precondition	The customer is authenticated and the cart has at least one item.
Postcondition	All items are removed from the cart.
Assumptions	The customer intentionally chooses to clear the cart.

Main Success Scenario:

68. The customer opens their cart.
69. The customer clicks 'Clear Cart'.
70. The system prompts for confirmation.
71. The customer confirms.
72. The system removes all cart items from the database.
73. The system displays an empty cart.

Use Case 16: Make Order

Identifier and Name	UC16 - Make Order
Initiator	Customer
Goal	The customer places an order for the products in their cart.
Precondition	The customer is authenticated, the cart is not empty, and payment info is available.
Postcondition	The order is placed, confirmed, and stored in the system.
Assumptions	The customer has a valid shipping address and payment method.

Main Success Scenario:

74. The customer proceeds to checkout from the cart.
75. The customer provides or confirms shipping address.
76. The customer selects a payment method.
77. The system validates stock availability for all cart items.
78. The system processes the payment.
79. The system creates the order record and sends a confirmation.
80. The system clears the cart.

Use Case 17: Get My Orders

Identifier and Name	UC17 - Get My Orders
Initiator	Customer
Goal	The customer views their personal order history.
Precondition	The customer is authenticated.
Postcondition	The customer's order history is displayed.
Assumptions	The customer has placed at least one order.

Main Success Scenario:

81. The customer navigates to the 'My Orders' page.
82. The system retrieves orders linked to the customer's account.
83. The system displays a list of orders with status, date, and total.

Use Case 18: List All Orders (Admin)

Identifier and Name	UC18 - List All Orders (Admin)
Initiator	Admin
Goal	The admin views a complete list of all platform orders.
Precondition	The admin is authenticated.
Postcondition	All orders are displayed in the admin panel.
Assumptions	At least one order exists in the system.

Main Success Scenario:

84. The admin navigates to the orders management section.
85. The system retrieves all orders from the database.
86. The system displays orders with customer info, items, status, and totals.

Use Case 19: Add to Wishlist

Identifier and Name	UC19 - Add to Wishlist
Initiator	Customer
Goal	The customer saves a product to their wishlist for future reference.
Precondition	The customer is authenticated and the product exists.
Postcondition	The product is added to the customer's wishlist.
Assumptions	The product is not already in the wishlist.

Main Success Scenario:

87. The customer views a product page.
88. The customer clicks the wishlist/heart icon.
89. The system adds the product to the customer's wishlist in the database.
90. The system confirms the addition with visual feedback.

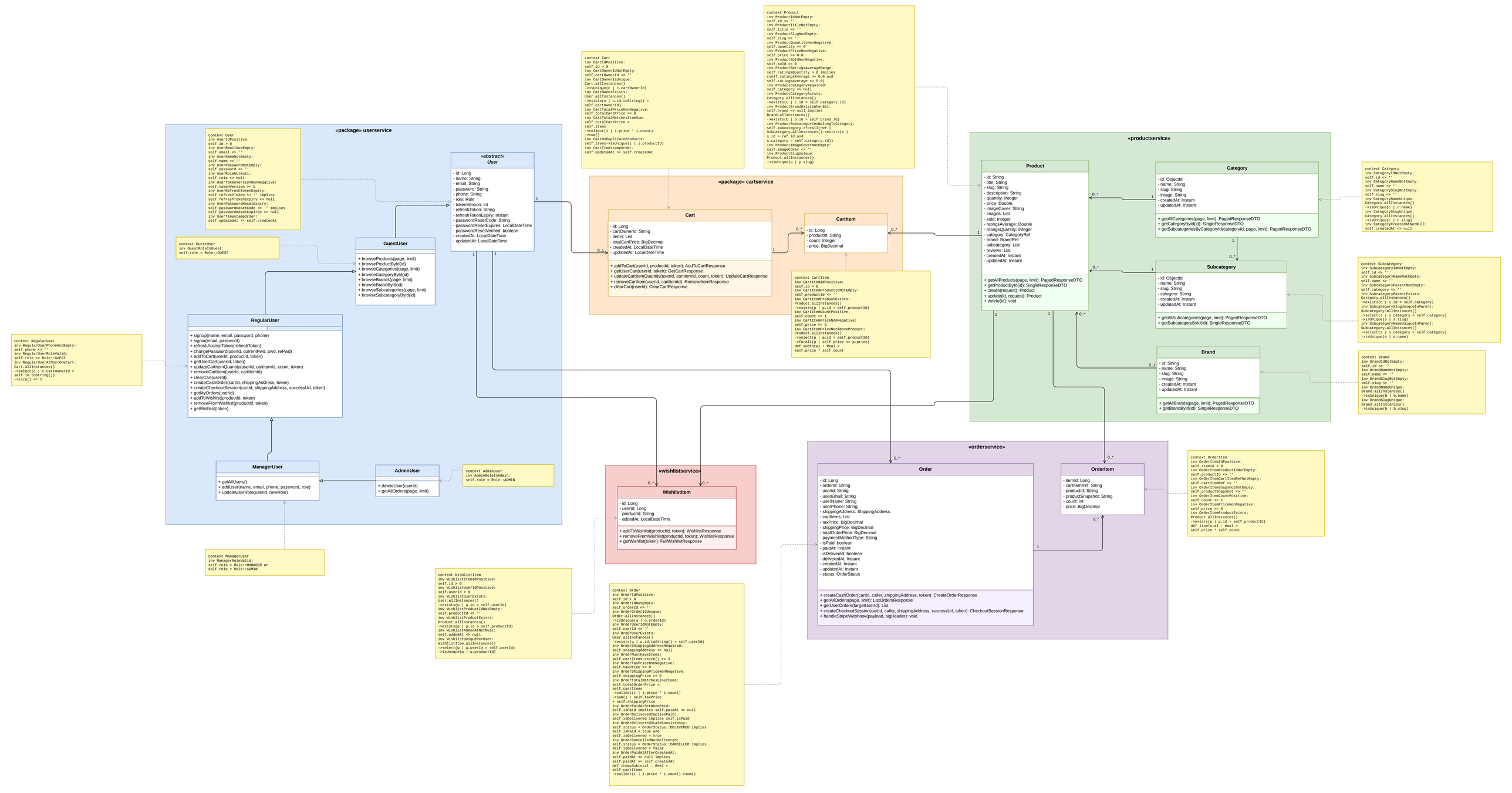
Use Case 20: Manage Users (Admin)

Identifier and Name	UC20 - Manage Users (Admin)

Initiator	Admin
Goal	The admin manages all user accounts: view, add, delete, and update roles.
Precondition	The admin is authenticated with admin privileges.
Postcondition	User records are created, deleted, or updated as requested.
Assumptions	The admin has valid data for new users or role changes.

Main Success Scenario:

91. The admin navigates to the user management panel.
92. The system retrieves and displays all registered users.
93. The admin can add a new user by filling in the registration form.
94. The admin can delete a user by selecting and confirming deletion.
95. The admin can update a user's role by selecting a role from a dropdown.
96. The system saves changes and confirms each action.


«package» userservice
«abstract»
User
- id: Long
- name: String
- email: String
- password: String
- phone: String
- role: Role
- tokenVersion: int
- refreshToken: String
- refreshTokenExpiry: Instant
- passwordResetCode: String
- passwordResetExpires: LocalDateTime
- passwordResetVerified: boolean
- createdAt: LocalDateTime
- updatedAt: LocalDateTime
context User
inv UserIdPositive:
self.id > 0
inv UserEmailNotEmpty:
self.email <> ''
inv UserNameNotEmpty:
self.name <> ''
inv UserPasswordNotEmpty:
self.password <> ''
inv UserRoleNotNull:
self.role <> null
inv UserTokenVersionNonNegative:
self.tokenVersion >= 0
inv UserRefreshTokenExpiry:
self.refreshToken <> '' implies
self.refreshTokenExpiry <> null
inv UserPasswordResetExpiry:
self.passwordResetCode <> '' implies
self.passwordResetExpires <> null
inv UserTimestampOrder:
self.updatedAt >= self.createdAt
context GuestUser
inv GuestRoleIsGuest:
self.role = Role::GUEST
GuestUser
+ browseProducts(page, limit)
+ browseProductById(id)
+ browseCategories(page, limit)
+ browseCategoryById(id)
+ browseBrands(page, limit)
+ browseBrandById(id)
+ browseSubcategories(page, limit)
+ browseSubcategoryById(id)
RegularUser
+ signup(name, email, password, phone)
+ signin(email, password)
+ refreshAccessToken(refreshToken)
+ changePassword(userId, currentPwd, pwd, rePwd)
+ addToCart(userId, productId, token)
+ getUserCart(userId, token)
+ updateCartItemQuantity(userId, cartItemId, count, token)
+ removeCartItem(userId, cartItemId)
+ clearCart(userId)
+ createCashOrder(cartId, shippingAddress, token)
+ createCheckoutSession(cartId, shippingAddress, successUrl, token)
+ getMyOrders(userId)
+ addToWishlist(productId, token)
+ removeFromWishlist(productId, token)
+ getWishlist(token)
context RegularUser
inv RegularUserPhoneNotEmpty:
self.phone <> ''
inv RegularUserRoleValid:
self.role <> Role::GUEST
inv RegularUserAtMostOneCart:
Cart.allInstances()
->select(c | c.cartOwnerId =
self.id.toString())
->size() <= 1
ManagerUser
+ getAllUsers()
+ addUser(name, email, phone, password, role)
+ updateUserRole(userId, newRole)
AdminUser
+ deleteUser(userId)
+ getAllOrders(page, limit)
context AdminUser
inv AdminRoleIsAdmin:
self.role = Role::ADMIN
context ManagerUser
inv ManagerRoleValid:
self.role = Role::MANAGER or
self.role = Role::ADMIN
context Cart
inv CartIdPositive:
self.id > 0
inv CartOwnerIdNotEmpty:
self.cartOwnerId <> ''
inv CartOwnerIdUnique:
Cart.allInstances()
->isUnique(c | c.cartOwnerId)
inv CartOwnerExists:
User.allInstances()
->exists(u | u.id.toString() =
self.cartOwnerId)
inv CartTotalPriceNonNegative:
self.totalCartPrice >= 0
inv CartTotalMatchesItemSum:
self.totalCartPrice =
self.items
->collect(i | i.price * i.count)
->sum()
inv CartNoDuplicateProducts:
self.items->isUnique(i | i.productId)
inv CartTimestampOrder:
self.updatedAt >= self.createdAt
«package» cartservice
Cart
- id: Long
- cartOwnerId: String
- items: List
- totalCartPrice: BigDecimal
- createdAt: LocalDateTime
- updatedAt: LocalDateTime
+ addToCart(userId, productId, token): AddToCartResponse
+ getUserCart(userId, token): GetCartResponse
+ updateCartItemQuantity(userId, cartItemId, count, token): UpdateCartResponse
+ removeCartItem(userId, cartItemId): RemoveItemResponse
+ clearCart(userId): ClearCartResponse
CartItem
- id: Long
- productId: String
- count: Integer
- price: BigDecimal
context CartItem
inv CartItemIdPositive:
self.id > 0
inv CartItemProductIdNotEmpty:
self.productId <> ''
inv CartItemProductExists:
Product.allInstances()
->exists(p | p.id = self.productId)
inv CartItemCountPositive:
self.count >= 1
inv CartItemPriceNonNegative:
self.price >= 0
inv CartItemPriceNotAboveProduct:
Product.allInstances()
->select(p | p.id = self.productId)
->forAll(p | self.price <= p.price)
def subtotal : Real =
self.price * self.count
context Product
inv ProductIdNotEmpty:
self.id <> ''
inv ProductTitleNotEmpty:
self.title <> ''
inv ProductSlugNotEmpty:
self.slug <> ''
inv ProductQuantityNonNegative:
self.quantity >= 0
inv ProductPriceNonNegative:
self.price >= 0.0
inv ProductSoldNonNegative:
self.sold >= 0
inv ProductRatingsAverageRange:
self.ratingsQuantity > 0 implies
(self.ratingsAverage >= 0.0 and
self.ratingsAverage <= 5.0)
inv ProductCategoryRequired:
self.category <> null
inv ProductCategoryExists:
Category.allInstances()
->exists(c | c.id = self.category.id)
inv ProductBrandExistsWhenSet:
self.brand <> null implies
Brand.allInstances()
->exists(b | b.id = self.brand.id)
inv ProductSubcategoriesBelongToCategory:
self.subcategory->forAll(ref |
Subcategory.allInstances()->exists(s |
s.id = ref.id and
s.category = self.category.id))
inv ProductImageCoverNotEmpty:
self.imageCover <> ''
inv ProductSlugUnique:
Product.allInstances()
->isUnique(p | p.slug)
«productservice»
Product
- id: String
- title: String
- slug: String
- description: String
- quantity: Integer
- price: Double
- imageCover: String
- images: List
- sold: Integer
- ratingsAverage: Double
- ratingsQuantity: Integer
- category: CategoryRef
- brand: BrandRef
- subcategory: List
- reviews: List
- createdAt: Instant
- updatedAt: Instant
+ getAllProducts(page, limit): PagedResponseDTO
+ getProductById(id): SingleResponseDTO
+ create(request): Product
+ update(id, request): Product
+ delete(id): void
Category
- id: ObjectId
- name: String
- slug: String
- image: String
- createdAt: Instant
- updatedAt: Instant
+ getAllCategories(page, limit): PagedResponseDTO
+ getCategoryById(id): SingleResponseDTO
+ getSubcategoriesByCategoryId(categoryId, page, limit): PagedResponseDTO
context Category
inv CategoryIdNotEmpty:
self.id <> ''
inv CategoryNameNotEmpty:
self.name <> ''
inv CategorySlugNotEmpty:
self.slug <> ''
inv CategoryNameUnique:
Category.allInstances()
->isUnique(c | c.name)
inv CategorySlugUnique:
Category.allInstances()
->isUnique(c | c.slug)
inv CategoryCreatedAtNotNull:
self.createdAt <> null
Subcategory
- id: ObjectId
- name: String
- slug: String
- category: String
- createdAt: Instant
- updatedAt: Instant
+ getAllSubcategories(page, limit): PagedResponseDTO
+ getSubcategoryById(id): SingleResponseDTO
context Subcategory
inv SubcategoryIdNotEmpty:
self.id <> ''
inv SubcategoryNameNotEmpty:
self.name <> ''
inv SubcategoryParentNotEmpty:
self.category <> ''
inv SubcategoryParentExists:
Category.allInstances()
->exists(c | c.id = self.category)
inv SubcategorySlugUniqueInParent:
Subcategory.allInstances()
->select(s | s.category = self.category)
->isUnique(s | s.slug)
inv SubcategoryNameUniqueInParent:
Subcategory.allInstances()
->select(s | s.category = self.category)
->isUnique(s | s.name)
Brand
- id: String
- name: String
- slug: String
- image: String
- createdAt: Instant
- updatedAt: Instant
+ getAllBrands(page, limit): PagedResponseDTO
+ getBrandById(id): SingleResponseDTO
context Brand
inv BrandIdNotEmpty:
self.id <> ''
inv BrandNameNotEmpty:
self.name <> ''
inv BrandSlugNotEmpty:
self.slug <> ''
inv BrandNameUnique:
Brand.allInstances()
->isUnique(b | b.name)
inv BrandSlugUnique:
Brand.allInstances()
->isUnique(b | b.slug)
«wishlistservice»
WishlistItem
- id: Long
- userId: Long
- productId: String
- addedAt: LocalDateTime
+ addToWishlist(productId, token): WishlistResponse
+ removeFromWishlist(productId, token): WishlistResponse
+ getWishlist(token): FullWishlistResponse
context WishlistItem
inv WishlistItemIdPositive:
self.id > 0
inv WishlistUserIdPositive:
self.userId > 0
inv WishlistUserExists:
User.allInstances()
->exists(u | u.id = self.userId)
inv WishlistProductIdNotEmpty:
self.productId <> ''
inv WishlistProductExists:
Product.allInstances()
->exists(p | p.id = self.productId)
inv WishlistAddedAtNotNull:
self.addedAt <> null
inv WishlistUniquePerUser:
WishlistItem.allInstances()
->select(w | w.userId = self.userId)
->isUnique(w | w.productId)
«orderservice»
Order
- id: Long
- orderId: String
- userId: String
- userEmail: String
- userName: String
- userPhone: String
- shippingAddress: ShippingAddress
- cartItems: List
- taxPrice: BigDecimal
- shippingPrice: BigDecimal
- totalOrderPrice: BigDecimal
- paymentMethodType: String
- isPaid: boolean
- paidAt: Instant
- isDelivered: boolean
- deliveredAt: Instant
- createdAt: Instant
- updatedAt: Instant
- status: OrderStatus
+ createCashOrder(cartId, caller, shippingAddress, token): CreateOrderResponse
+ getAllOrders(page, limit): ListOrdersResponse
+ getUserOrders(targetUserId): List
+ createCheckoutSession(cartId, caller, shippingAddress, successUrl, token): CheckoutSessionResponse
+ handleStripeWebhook(payload, sigHeader): void
OrderItem
- itemId: Long
- cartItemRef: String
- productId: String
- productSnapshot: String
- count: int
- price: BigDecimal
context OrderItem
inv OrderItemIdPositive:
self.itemId > 0
inv OrderItemProductIdNotEmpty:
self.productId <> ''
inv OrderItemCartItemRefNotEmpty:
self.cartItemRef <> ''
inv OrderItemSnapshotNotEmpty:
self.productSnapshot <> ''
inv OrderItemCountPositive:
self.count >= 1
inv OrderItemPriceNonNegative:
self.price >= 0
inv OrderItemProductExists:
Product.allInstances()
->exists(p | p.id = self.productId)
def lineTotal : Real =
self.price * self.count
context Order
inv OrderIdPositive:
self.id > 0
inv OrderIdNotEmpty:
self.orderId <> ''
inv OrderOrderIdUnique:
Order.allInstances()
->isUnique(o | o.orderId)
inv OrderUserIdNotEmpty:
self.userId <> ''
inv OrderUserExists:
User.allInstances()
->exists(u | u.id.toString() = self.userId)
inv OrderShippingAddressRequired:
self.shippingAddress <> null
inv OrderMustHaveItems:
self.cartItems->size() >= 1
inv OrderTaxPriceNonNegative:
self.taxPrice >= 0
inv OrderShippingPriceNonNegative:
self.shippingPrice >= 0
inv OrderTotalMatchesLineItems:
self.totalOrderPrice =
self.cartItems
->collect(i | i.price * i.count)
->sum() + self.taxPrice
+ self.shippingPrice
inv OrderPaidAtSetWhenPaid:
self.isPaid implies self.paidAt <> null
inv OrderDeliveredImpliesPaid:
self.isDelivered implies self.isPaid
inv OrderDeliveredStateConsistency:
self.status = OrderStatus::DELIVERED implies
self.isPaid = true and
self.isDelivered = true
inv OrderCancelledNotDelivered:
self.status = OrderStatus::CANCELLED implies
self.isDelivered = false
inv OrderPaidAtAfterCreatedAt:
self.paidAt <> null implies
self.paidAt >= self.createdAt
def itemsSubtotal : Real =
self.cartItems
->collect(i | i.price * i.count)->sum()
1
0..1
0..*
1..*